



- Instruction-Finetuned Vision-Language-Action Model with Locomotion-Aware Chain-of-Thought

Amitava Das^{*}

^{*}Pragya Lab, BITS Pilani Goa, India

Abstract

Vision-language-action (VLA) models have recently emerged as a promising paradigm for grounding language in embodied control. However, most existing VLA systems are developed for *manipulation-centric* or clean indoor settings, where the principal challenge is task completion rather than **safe mobility under real-world uncertainty**. In this work, we introduce **Pragya-VLA**, an **instruction-finetuned quadruped VLA framework** designed around four core pillars: **instruction following, locomotion-aware chain-of-thought reasoning, Hindi language grounding, and deployment in dense, uneven, and semi-structured Indian terrains**. Our central premise is that quadruped robots operating in realistic environments must do more than interpret a command: they must determine *whether, how, and how safely* that command can be executed under constraints induced by terrain, occlusion, crowds, and uncertainty.

First, Pragya-VLA is formulated as an **instruction-tuned model** that maps natural language commands into a structured executable intent representation comprising **task goals, hard constraints, soft preferences, and conditional triggers**. This enables the system to follow not only direct commands, but also safety-conditioned instructions such as avoiding slippery ground, maintaining distance from pedestrians, stopping under uncertainty, or preferring safer but longer routes. **Second**, rather than directly mapping observations and instructions to actions, Pragya-VLA inserts a **locomotion-aware chain-of-thought** in the form of a **structured deliberation bottleneck** that performs intermediate inference over **terrain traversability, full-body feasibility, hidden environmental state, and multi-factor safety risk**. Concretely, given visual observations, proprioceptive history, and parsed intent, the model predicts a latent deliberation state that parameterizes a local traversability field, a body-swept feasibility score over candidate motion primitives, and a risk functional aggregating collision, slip, clearance, social-proximity, and uncertainty penalties. Action selection is then cast as **constrained utility maximization**, where the chosen behavior maximizes task satisfaction while respecting instruction constraints and minimizing physical and social risk. In this way, chain-of-thought is not treated as a free-form textual explanation, but as a **decision-theoretic latent process** specialized for quadruped mobility.

Third, Pragya-VLA supports both **English and Hindi instructions**, allowing semantically equivalent commands across the two languages to be aligned into a common intent space through an explicit **cross-lingual consistency objective**. This ensures that multilingual instructions induce comparable deliberation states and downstream actions rather than merely sharing a superficial text interface. **Fourth**, we explicitly target **Indian deployment conditions**, including *uneven outdoor surfaces, cluttered public pathways, narrow passages, ramps, loose gravel, wet floors, mixed indoor-outdoor transitions, and crowd-heavy navigation*, which remain significantly underrepresented in existing VLA benchmarks. These conditions are particularly challenging for quadrupeds because low-viewpoint perception and egocentric occlusion make safe locomotion strongly dependent on temporal memory. To address this, Pragya-VLA incorporates a **memory-conditioned hidden-state estimator** that reconstructs partially observed terrain and obstacle geometry from visual and proprioceptive history before passing this estimate into the deliberation bottleneck.

Technically, Pragya-VLA combines **(i)** a constraint-grounded instruction parser, **(ii)** a memory-conditioned hidden-state inference module, **(iii)** a locomotion-aware deliberation bottleneck for traversability, feasibility, and risk prediction, and **(iv)** a safe action head that selects among both *motion* and *abstention* behaviors such as **proceed, slow, reroute, stop, retreat, and defer**. We train the model with a **multi-objective instruction fine-tuning objective** that jointly supervises instruction parsing, deliberation-state prediction, feasibility estimation, risk calibration, Hindi-English intent consistency, abstention correctness, and action generation. We evaluate Pragya-VLA on instruction-conditioned quadruped navigation and patrol tasks spanning **multilingual instruction following, terrain-aware reasoning, crowd-sensitive movement, and uncertainty-aware stopping**. Across these settings, Pragya-VLA improves **instruction fidelity, safety, robustness, and risk-sensitive abstention** over direct policy baselines and generic reasoning-augmented variants. **Overall, our results argue that the next frontier of VLA lies not merely in better action generation, but in unifying instruction tuning, embodied deliberation, multilingual grounding, and terrain-aware safety within a single mathematically structured control framework for real-world quadruped robotics.**

1 Kimodo-Based Curation of a 5,000-Prompt Multilingual Motion-Instruction Corpus for Hindi, Bengali, and Telugu

We propose a rigorous pipeline for constructing a **5,000-prompt multilingual motion-instruction corpus** using **Kimodo** as the primary motion authoring and motion validation engine, with **Hindi, Bengali, and Telugu** as the first three Indian target languages. The core philosophy is to treat Kimodo not merely as a text-to-motion generator, but as a **structured semantic filter** that helps determine which English prompts are sufficiently precise, controllable, distinct, and embodiment-compatible to deserve multilingual expansion. In this design, English prompts are first generated from a tightly controlled taxonomy, then stress-tested through Kimodo-based motion realization, and only afterwards translated into Indian languages. This ordering is fundamental. If translation happens too early, ambiguities in the English source cascade into all derived languages. By contrast, validating the English source space first ensures that the multilingual corpus inherits *motion-grounded semantic stability, controlled compositionality, and robot-executable phrasing*.

The resulting resource is intentionally positioned between two existing paradigms. On one side are classical text-to-motion datasets, which often prioritize expressive human motion but do not provide enough structure for robotics-style command following. On the other side are vision-and-language navigation or embodied instruction datasets, which provide route descriptions or task-oriented dialogue but are not optimized for clean motion primitive grounding. Our corpus occupies the middle layer: it is a **multilingual motion-instruction dataset with controlled navigation semantics**, designed to support downstream humanoid and legged robot instruction following, multilingual motion retrieval, motion-conditioned policy learning, simulation-based validation, and eventually real-world embodied deployment.

1.1 Design Objective, Scope, and Representational Philosophy

The dataset objective is to construct **5,000 unique motion intents**, where each intent is represented by:

1. a **canonical English instruction** that preserves explicit semantic structure,
2. a **robot-natural English instruction** that is concise and execution-oriented,
3. a **literal translation** into Hindi, Bengali, and Telugu,
4. a **robot-natural translation** into Hindi, Bengali, and Telugu,
5. a structured metadata block that captures motion family, compositional structure, constraint type, embodiment compatibility, safety/risk, ambiguity profile, translation status, and motion validation status.

This representation is deliberately redundant. The canonical English form serves as the semantic source of truth; the robot-natural English form normalizes execution style; the literal translations preserve cross-lingual faithfulness; and the robot-natural translations serve downstream embodied systems where concise imperatives are often better than literary or conversational phrasing. Thus, the corpus is not a simple multilingual paraphrase resource. It is a **hierarchically normalized motion semantics dataset**.

The prompts should be designed to cover the following broad semantic territories:

- **basic locomotion** (move, turn, stop, sidestep, back up),
- **posture transitions** (crouch, kneel, rise, lean, bow),
- **constraint-conditioned motion** (waypoints, dense paths, obstacles, end-pose targets),
- **landmark-grounded motion** (relative to doors, markers, tables, gates, walls, corridors),
- **safety and abstention behavior** (slow down, stop, inspect, wait, avoid),
- **socially aware motion** (pause near person, keep distance, wave, yield),
- **long-horizon compositional tasks** with finite and evaluable subgoal chains.

The prompt space should *exclude* open-ended storytelling, metaphorical expressions, highly culture-bound idioms, non-executable flourish, and instructions whose semantics depend primarily on world knowledge rather than on movement structure. In other words, the corpus should privilege **semantic executability** over stylistic creativity.

1.2 Rigid Seven-Tier Structure for 5,000 Prompts

A central requirement of this corpus is that its tier structure must be **rigid, exhaustive, and mutually exclusive**. To achieve that, we partition the inventory into seven formally defined tiers, each identified by the *highest-complexity property* present in the instruction. This avoids a common failure mode in instruction datasets, where simple commands, path-conditioned tasks, safety interventions, and long-horizon plans are mixed into a single flat pool. Our proposed tier design is shown in Table 1.

Tier	Name	Count	Definition	Typical Example	Primary QC Focus
T1	Atomic primitives	1,200	Single motion intent, no explicit modifier except the action itself.	<i>Stop. Turn left. Crouch. Raise right arm.</i>	lexical clarity, semantic uniqueness
T2	Parameterized atomics	800	Single action with exactly one modifier: count, direction, side, speed, or duration proxy.	<i>Walk forward three steps. Turn right slowly.</i>	parameter preservation, modifier consistency
T3	Short composites	1,000	Two or three ordered actions in compact sequence.	<i>Walk forward and turn left. Crouch, move ahead, then stand up.</i>	action order, temporal coherence
T4	Constraint-conditioned prompts	700	Prompt meaning depends on a path, waypoint, obstacle, keyframe, or end-state constraint.	<i>Follow the curved path and stop at the marker.</i>	constraint compliance, path fidelity
T5	Landmark/navigation-context prompts	600	Motion described relative to landmarks such as doors, tables, gates, walls, or corridors.	<i>Move to the red door and stop.</i>	landmark grounding, reference clarity
T6	Safety / abstention / social prompts	400	Includes caution, yielding, waiting, inspection, uncertainty, or human-aware behavior.	<i>If the path is blocked, stop and wait.</i>	safety semantics, abstention fidelity
T7	Long-horizon tasks	300	Four to six ordered subgoals with mixed movement, landmarks, and simple interaction.	<i>Exit the room, turn right, follow the corridor, pass the chair, and stop at the door.</i>	long-range consistency, subgoal completion

Table 1: Rigid seven-tier decomposition of the proposed 5,000-prompt Kimodo-based multilingual motion-instruction corpus. Each prompt is assigned according to its highest-complexity property.

These seven tiers should be interpreted operationally as follows.

T1: Atomic primitives. This tier contains the smallest executable units of motion semantics. These are single-action commands with no additional parameterization beyond the core action itself. Typical examples include *Stop*, *Turn left*, *Crouch*, *Wave*, or *Step back*. A prompt belongs to T1 only if it contains exactly one action head and no overt count, speed modifier, spatial landmark, or second action. T1 is not trivial: it is foundational. It defines the basic motor lexicon out of which all higher tiers are composed.

T2: Parameterized atomics. This tier contains instructions with one core action head and exactly one modifier. That modifier may encode step count, direction, body side, speed, or a duration proxy. For instance, *Walk forward three steps*, *Raise your right arm*, and *Turn left slowly* are valid T2 prompts. If two action heads appear, the prompt must move to T3. T2 is especially important because it introduces structured parameter binding without yet entering compositional sequencing.

T3: Short composites. This tier captures two- or three-step compositional motion instructions. These are compact, ordered sequences such as *Walk forward and turn left, Crouch, move ahead, then stand up, or Turn right, wave, and stop*. T3 is the first tier where temporal ordering becomes central. It is therefore the first tier where motion review must check not just action presence, but *ordering correctness* and *transition plausibility*.

T4: Constraint-conditioned prompts. This tier contains instructions whose semantics are not fully determined by the text alone, but rely on an external motion constraint payload. These constraints may be dense paths, waypoint lists, obstacle relations, end-pose requirements, or keyframe specifications. Examples include *Follow the curved path and stop at the marker, Move to waypoint A, then waypoint B, or Duck under the low bar and continue*. This tier is central to Kimodo-based curation because Kimodo's value lies partly in its ability to realize motion from text under such constraints.

T5: Landmark/navigation-context prompts. This tier contains instructions grounded relative to scene entities such as doors, gates, ramps, markers, walls, corridors, tables, or chairs. These prompts are still motion-centric, but they introduce light environmental context. Examples include *Move to the gate and stop, Turn left at the marker, or Pass the chair and stop near the wall*. These prompts benefit strongly from inspiration drawn from embodied navigation corpora, but must remain shorter and more execution-oriented than typical route descriptions.

T6: Safety, abstention, and social prompts. This tier is designed for deployment-sensitive motion semantics. It includes instructions such as *Move slowly over the uneven surface, If the path is blocked, stop and wait, Inspect the obstacle before moving, or Slow down near the person*. The defining property here is not merely movement, but **safety-aware behavior**. This tier is especially valuable for future robot deployment, because it introduces conditional stopping, caution, and human-aware conduct directly into the text layer.

T7: Long-horizon tasks. This tier contains compact but extended tasks with four to six sequential subgoals. These prompts remain finite and structured; they are not free-form narratives. A valid example is: *Exit the room, turn right, follow the corridor, pass the chair, and stop at the door*. T7 represents the upper bound of compositional complexity in version 1 of the corpus. It also serves as a natural bridge to navigation-style instruction following, but without abandoning motion specificity.

1.3 Prompt Grammar and English Candidate Generation

The 5,000 English prompts should not be authored manually one by one. Instead, they must be produced from a **controlled prompt grammar** that balances coverage with discipline. The grammar should define closed or semi-closed inventories over action heads, parameter slots, landmarks, path types, obstacles, and safety conditions. This gives the dataset a principled combinatorial structure and prevents uncontrolled drift in style.

Action lexicon. The core action vocabulary should include:

- **locomotion:** walk, move, go, turn, sidestep, back up, stop,
- **posture:** crouch, kneel, stand, rise, lean, bow,
- **obstacle handling:** duck, step over, go around, move past,
- **gesture / body orientation:** wave, point, raise arm, lower arm, face left, face right.

Modifier slots. Parameterized prompts should be built using:

- **direction:** forward, backward, left, right, toward, away from, around,
- **count:** one, two, three, five steps,
- **speed / caution:** slowly, carefully, steadily,
- **body side:** left arm, right arm, left side, right side,
- **landmarks:** door, gate, marker, chair, table, wall, corridor, ramp, cone, barrier,

- **path / constraint types:** straight path, curved path, zigzag path, waypoint A, waypoint B, obstacle, low bar, gap,
- **conditionals:** if blocked, if unsafe, before crossing, when clear, inspect first.

Grammar templates. The English prompt generator should instantiate patterns such as

ACTION, ACTION + DIRECTION, ACTION + COUNT, ACTION + SPEED,

ACTION₁ + ACTION₂, ACTION₁ + ACTION₂ + ACTION₃,

FOLLOW + PATH_TYPE, MOVE + LANDMARK_RELATION,

IF + CONDITION + ACTION, SEGMENT₁ + SEGMENT₂ + SEGMENT₃.

Operationally, the right workflow is to generate approximately **6,000–6,500 candidate English prompts**, then deduplicate, collapse near-synonymous variants, and downselect to the final 5,000 according to the fixed tier quotas. This over-generation stage is important because the initial grammar will inevitably produce some semantically redundant, stylistically awkward, or motion-ambiguous prompts. Over-generation therefore acts as a reservoir for later refinement rather than a wasteful step.

1.4 Canonical English and Robot-Natural English

Each motion intent should be stored in two English forms:

1. **Canonical English**, which preserves explicit structure and is as close as possible to a semantically transparent source form.
2. **Robot-natural English**, which rewrites the prompt into a concise imperative intended for downstream instruction following.

For example:

Canonical: *Walk forward three steps and then turn left.*

Robot-natural: *Move forward three steps, then turn left.*

This distinction is valuable for two reasons. First, it separates *semantic traceability* from *execution style*. Second, it improves multilingual consistency, because literal translations from expressive English often sound stiff or misleading when mapped into Indian-language command forms. The robot-natural English prompt acts as a normalization layer that reduces such drift. In practice, the robot-natural form should be short, imperative, and free from optional stylistic material. It should also avoid pronouns, discourse markers, and polite flourishes unless such forms are explicitly being studied.

1.5 Kimodo-Based Motion Validation: Semantic Filtering Before Translation

Kimodo should enter the pipeline only after the English prompt bank has been generated and tiered, but before any translation begins. The correct order is:

1. define the tier taxonomy,
2. generate candidate English prompts,
3. assign metadata and remove duplicates,
4. run Kimodo-based validation on representative subsets,
5. revise unstable prompts,
6. freeze the English source set,

7. translate into Hindi, Bengali, and Telugu.

The purpose of Kimodo here is not merely to synthesize attractive motion clips. Its deeper purpose is to function as a **semantic validator**. A prompt should be retained only if its generated motion is:

- **plausible**,
- **distinct** from nearby prompts,
- **constraint-compliant** when constraints are provided,
- **compatible** with the target embodiment,
- **stable enough** to justify multilingual replication.

Examples of prompts that should be rewritten or removed include:

- prompts with vague spatial relations, e.g., *Move near the chair*,
- prompts with multiple equally plausible realizations, e.g., *Shift position slightly*,
- prompts whose semantics depend on unsupported affordances,
- prompts that systematically cause implausible transitions or severe motion artifacts.

We recommend tier-dependent motion review rates:

- **T1/T2**: validate 10–15%,
- **T3/T4**: validate 20–25%,
- **T5/T6/T7**: validate 30–40%.

The rationale is simple: ambiguity and kinematic fragility increase with tier complexity. T1 and T2 are short and local. T4 involves explicit constraint payloads. T6 and T7 introduce social, safety, and long-horizon reasoning, which are precisely the settings where latent ambiguity becomes most expensive.

1.6 Embodiment Tags and Technical Specifications

Because the corpus is intended for future embodied AI and robotics use, every prompt must carry an explicit **embodiment profile**. At minimum, the schema should include:

- `target_body`: {g1, soma, generic_humanoid},
- `locomotion_type`: {walk, turn, pose, gesture, mixed},
- `constraint_type`: {none, waypoint, dense_path, keyframe, obstacle, end_pose},
- `robot_feasibility`: {safe, needs_review, unsafe},
- `terrain_dependence`: {none, flat_only, mild_obstacle, scene_dependent},
- `contact_complexity`: {low, medium, high},
- `sequence_duration_s`: numeric,
- `target_fps`: numeric,
- `requires_postprocess`: boolean,
- `validation_status`: {unverified, kimodo_ok, postprocessed_ok, human_verified}.

A practical first-pass technical configuration is:

- **motion frame rate**: 30 fps,

- **short prompts (T1/T2):** 2.5–4 seconds,
- **medium prompts (T3/T5):** 4–6 seconds,
- **constraint-heavy prompts (T4/T6):** 5–8 seconds,
- **long-horizon prompts (T7):** 8–10 seconds,
- **default review artifact:** motion clip + metadata + constraint payload,
- **postprocess checks:** foot sliding, abrupt jerk, end-pose mismatch, path deviation, waypoint miss, self-collision, obstacle penetration.

It is also useful to define a **motion family tag** orthogonal to the tier:

- basic locomotion,
- posture transition,
- obstacle negotiation,
- path following,
- landmark-conditioned movement,
- social gesture,
- safety intervention,
- long-horizon mixed.

This allows experiments that group prompts by motor semantics rather than only by linguistic complexity.

1.7 Translation Protocol for Hindi, Bengali, and Telugu

After the English source bank is frozen, every prompt should be translated into **Hindi, Bengali, and Telugu** in two forms:

1. **Literal translation**, preserving lexical structure and action order as closely as possible.
2. **Robot-natural translation**, preserving execution meaning while making the command concise, imperative, and natural for robot-facing use.

For example, consider the robot-natural English instruction:

Move forward three steps, then turn left.

A corresponding multilingual set may be:

Hindi (literal) : तीन कदम आगे चलो, फिर बाएँ मुड़ो।

Hindi (robot-natural) : तीन कदम आगे बढ़ो, फिर बाईं ओर मुड़ो।

Bengali (literal) : তিন ধাপ সামনে হাঁটো, তারপর বাঁ দিকে ঘুরো।

Bengali (robot-natural) : তিন ধাপ সোজা এগিয়ে যাও, তারপর বাঁ দিকে ঘুরো।

Telugu (literal) : మూడు అడుగులు ముందుకు నడవండి, తరువాత ఎడమవైపు తిరగండి।

Telugu (robot-natural) : మూడు అడుగులు ముందుకు వెళ్లి, తరువాత ఎడమవైపు తిరగండి।

The translation process should explicitly verify four properties:

1. **Action preservation:** every action head survives translation.
2. **Order preservation:** action order remains unchanged.

3. **Direction preservation:** left/right/toward/away remain correct.
4. **Safety preservation:** caution, abstention, and inspection semantics remain equally strong.

In addition, each translated item should carry metadata such as:

- translation_style: {literal, robot_natural},
- ambiguity_flag,
- ambiguity_type,
- requires_scene_context,
- requires_body_reference,
- safe_for_direct_execution.

These fields are not auxiliary; they are analytically central. They allow later study of whether translation-induced variation correlates with motion-generation instability, multilingual instruction-following degradation, or safety-semantic erosion.

1.8 Quality Control Pipeline

The full corpus should be filtered through a three-stage quality-control pipeline.

Pass A: Linguistic QC. This pass checks grammar, clarity, tier consistency, duplication, unintended synonym collisions, and hidden conditionals. Prompts that differ lexically but not semantically should be merged before translation. Prompts that accidentally drift across tiers should be reassigned or rewritten.

Pass B: Motion QC. This pass checks whether the Kimodo-generated motion matches the intended semantics. For constraint-conditioned prompts, it also checks path or waypoint fidelity. For posture or gesture prompts, it checks end-state realism. For all prompts, it checks severe artifacts such as foot sliding, penetration, unstable transitions, and incorrect action ordering.

Pass C: Multilingual QC. This pass checks whether the translations preserve action sequence, directional terms, safety or abstention semantics, and landmark identity. It also checks whether robot-natural rewrites remain concise imperatives rather than drifting into descriptive or conversational forms.

To control annotation effort while maintaining rigor, review intensity should vary by tier:

- T1/T2: lowest review burden,
- T3/T4: medium review burden,
- T5/T6/T7: highest review burden.

Reviewer roles should ideally include:

- one motion reviewer,
- one bilingual reviewer per target language,
- one adjudicator on overlapping subsets.

The overlapping subset is especially useful for computing reviewer agreement on left/right preservation, count fidelity, action-order consistency, and safety-language preservation.

1.9 Dataset Schema and Release Format

The final release should contain:

1. a **JSONL corpus** with all prompt-level metadata,
2. associated **motion assets**,
3. optional **visualization clips** for audit,
4. **constraint payloads** for T4 prompts,
5. a **tier annotation manual**,
6. a **translation guideline document**.

A representative schema is:

```
{
  "prompt_id": "kimodo_000421",
  "tier": "T4",
  "subtier": "waypoint_conditioned",
  "instruction_en": {
    "canonical": "Follow the curved path and stop at the marker.",
    "robot_natural": "Follow the curved path and stop at the marker."
  },
  "translation_hi": {
    "literal": "...",
    "robot_natural": "..."
  },
  "translation_bn": {
    "literal": "...",
    "robot_natural": "..."
  },
  "translation_te": {
    "literal": "...",
    "robot_natural": "..."
  },
  "motion_family": "path_following",
  "primitive_tags": ["follow_path", "stop"],
  "constraint_type": "dense_path",
  "landmarks": ["marker"],
  "safety_type": "none",
  "ambiguity_flag": false,
  "embodiment": {
    "target_body": "g1",
    "robot_feasibility": "safe"
  },
  "kimodo": {
    "validated": true,
    "constraint_payload_id": "path_023",
    "motion_asset": "motions/kimodo_000421.npz"
  }
}
```

The recommended split is:

- **train:** 4,000 prompts,

- **dev:** 500 prompts,
- **test:** 500 prompts.

The split must be stratified by tier. In addition, the test set should slightly over-sample T4, T6, and T7 because these tiers carry the richest information about constraint handling, safety semantics, and long-horizon instruction following.

1.10 Step-by-Step Execution Plan

The full dataset construction should proceed in the following order:

1. **Freeze the seven-tier taxonomy** and define all subtiers.
2. **Specify the prompt grammar**, slot vocabularies, and prohibited patterns.
3. **Generate 6,000–6,500 English candidates** through template instantiation.
4. **Deduplicate and downselect to 5,000**, matching the target tier counts.
5. **Assign metadata** including primitive tags, motion family, constraint type, embodiment tags, and initial ambiguity flags.
6. **Run Kimodo validation on representative subsets** tier by tier.
7. **Rewrite or remove unstable prompts** whose realizations are ambiguous, implausible, or low-value.
8. **Freeze the English source set** in canonical and robot-natural form.
9. **Translate each prompt into Hindi, Bengali, and Telugu** in literal form.
10. **Produce robot-natural translations** for each target language.
11. **Run linguistic QC and multilingual QC** using bilingual reviewers.
12. **Perform targeted motion review** on high-risk tiers and sampled lower-risk tiers.
13. **Assemble JSONL, motion assets, and constraint payloads.**
14. **Construct train/dev/test splits** with tier stratification.
15. **Release the corpus** with guidelines, examples, and documentation.

1.11 Summary

The proposed resource is strongest when viewed not as a loose collection of translated text-to-motion prompts, but as a **controlled multilingual motion-instruction corpus** in which English source prompts are systematically generated, Kimodo is used to validate motion semantics and constraint realizability, and the final Hindi, Bengali, and Telugu variants are explicitly engineered for robot-style execution. The rigid tier structure, grammar-based source generation, embodiment-aware metadata, two-stage translation procedure, and tier-specific quality control together make the corpus scalable, auditable, and suitable for future multilingual embodied AI systems. The natural next deliverable after this section is a **master taxonomy sheet** enumerating all seven tiers, their subtiers, allowed and forbidden grammar patterns, representative examples, reviewer checks, and expected motion validation criteria.

2 Plan 2: Existing-Dataset Curation, Multilingual Translation, Robotization, and Simulation

This section presents the second data-construction strategy for Pragya-style multilingual embodied instruction learning: **existing-datasets** → **translation** → **robotization** → **simulation**. In contrast to the Kimodo-first pipeline, which begins by designing a controlled motion-prompt inventory and then validating it through motion generation, Plan 2 begins with **already available embodied navigation and instruction datasets**, extracts their most useful English instruction forms, translates them into

Indian languages, rewrites them into execution-oriented robot commands, and finally tests whether the translated and robotized versions preserve the original route semantics under simulation.

The main advantage of this plan is that it inherits **human-authored route language** and **task-grounded embodied supervision** from mature benchmarks. In particular, **RxR** is unusually valuable because it is already multilingual, contains **126k navigation instructions** and **126k navigation-following demonstrations**, and includes **English, Hindi, and Telugu** aligned instruction resources together with follower metrics, pose traces, and additional auxiliary assets (Ku et al., 2020; rxr). **R2R** remains the canonical compact English navigation benchmark with roughly **22k instructions** averaging about **29 words**, and it remains tightly coupled to the Matterport simulation ecosystem (Anderson et al., 2018; mat). **FGR2R** is particularly important for decomposition, since it enriches route instructions with **sub-instructions and sub-path alignments**, which is exactly what is needed before multilingual translation and robotization (Hong et al., 2020). More recent systems such as **NaVILA** provide an embodiment bridge toward legged navigation by converting natural language into **mid-level language actions with spatial structure**, making them particularly relevant for the robotization phase (Cheng et al., 2025; nav). The objective of Plan 2 is therefore not merely to translate existing benchmarks, but to **systematically transform them into a robotics-ready multilingual corpus** while preserving route semantics, landmark grounding, safety cues, and action order.

2.1 Overall Goal and Corpus Construction Philosophy

The goal of Plan 2 is to construct a multilingual route-and-action corpus in which each source instruction is represented through multiple layers of normalization and translation. For every selected instruction from an existing dataset, we create:

1. the **raw source instruction**,
2. a **canonical normalized English instruction**,
3. a **robot-natural English instruction**,
4. a **literal Hindi translation**,
5. a **robot-natural Hindi translation**,
6. a **literal Bengali translation**,
7. a **robot-natural Bengali translation**,
8. a **literal Telugu translation**,
9. a **robot-natural Telugu translation**,
10. a structured metadata block containing **subgoals, landmark references, turn events, stop conditions, safety cues, source dataset provenance, and simulation status**.

This layered representation has three benefits. First, it preserves provenance: every translated instruction can be traced back to a concrete original dataset item. Second, it makes multilingual quality control measurable: one can compare raw, canonical, robotized, and translated forms within the same alignment scaffold. Third, it allows simulation-based equivalence testing: one can ask whether the translated and robotized instructions induce the *same navigational behavior class* as the original English instruction under a shared simulator and route specification.

The key conceptual distinction of Plan 2 is that it treats translation as only one step in a longer **semantic conversion pipeline**. Existing dataset instructions are often too verbose, too spoken, too dialogic, or too environment-specific to be directly usable as robot commands. Hence, the pipeline must explicitly include **normalization** and **robotization** before simulation.

Resource	Type	What It Contributes	Primary Role in Plan 2	Official Link
RxR (Ku et al., 2020; rxr)	Indoor VLN	126k multilingual instructions (English, Hindi, Telugu), follower annotations, pose traces, dense grounding	Main multilingual backbone; translation anchor; simulation equivalence source	https://github.com/google-research-datasets/RxR
R2R (Anderson et al., 2018; mat)	Indoor VLN	Compact English route instructions; standard Matterport linkage	Clean English seed set; baseline simulator-aligned routes	https://github.com/peteanderson80/Matterport3DSimulator
FGR2R (Hong et al., 2020)	Fine-grained VLN	Sub-instructions and sub-path alignment	Instruction decomposition before translation and robotization	https://github.com/YicongHong/Fine-Grained-R2R
Touchdown (Chen et al., 2019; tou)	Outdoor VLN	Street-scale English navigation and spatial reasoning	Outdoor phrasing augmentation; urban landmark language	https://github.com/lil-lab/touchdown
CVDN (Thomason et al., 2019; cvd)	Dialog navigation	Human-human navigation dialogs and trajectories	Clarification and interaction layer for future multilingual extensions	https://github.com/murray/cvdn
TEACH (Padmakumar et al., 2022; tea)	Embodied dialog/task	Task-oriented embodied instructions in households	Navigation-plus-action phrasing; action-rich compositional extensions	https://github.com/alexa/teach
REVERIE (Qi et al., 2020; rev)	Object-grounded navigation	Referring-expression-based destination grounding	Object-centric target language and destination descriptions	https://github.com/YuankaiQi/REVERIE
SOON / FAO (soo)	Object navigation	Destination-by-description language from arbitrary starts	Destination-style instruction augmentation	https://github.io/scenario-oriented-object-nav
NaVILA (Cheng et al., 2025; nav)	Legged VLA navigation	Mid-level language actions for legged navigation	Embodiment bridge; robotization style reference	https://github.io/navila-bot
QUAR-VLA / QUARD (Ding et al., 2024; qua)	Quadruped VLA	Quadruped-native instruction/task schema	Quadruped task design reference; future bridging	https://github.io/quart-robot
VLN-CE (vln)	Continuous VLN sim	Continuous-control port of R2R in Habitat	Stronger physical simulation for translated instructions	https://github.com/jacobkrantz/VLN-CE
VLN-PE (Wang et al., 2025)	Embodiment evaluation	Humanoid, quadruped, and wheeled evaluation settings	Physical realism and embodiment-gap testing	Project linked from paper page
LHPR-VLN (Song et al., 2025; lhp)	Long-horizon VLN	3,260 long-horizon tasks with complex task structure	Stress-testing long translated instructions	https://huggingface.co/datasets/Starry123/LHPR-VLN
VLN-Trans (Zhang and Kordjamshidi, 2023)	Instruction transformation	Agent-friendly translation of navigation language	Decomposition and robotization methodology reference	ACL Anthology paper page

Table 2: Key resources for Plan 2 and their operational role in the pipeline. The official links are listed to support direct dataset or code access.

2.2 Dataset Landscape and Resource Roles

The relevant datasets for Plan 2 do not all serve the same purpose. Some are best suited as **multi-lingual anchors**, some as **compact English instruction sources**, some as **decomposition aids**, some as **outdoor landmark language sources**, and some as **simulation or evaluation platforms**. Table 2 summarizes the most important resources and their role in the Plan 2 pipeline.

The most important design decision is that these resources should not be merged blindly. Instead, each source should be assigned a function:

- **RxR** serves as the *main multilingual backbone* because it already contains aligned English, Hindi, and Telugu instruction resources and rich route-following metadata (Ku et al., 2020; rxr).
- **R2R** serves as the *clean English seed set* because its instructions are shorter and more canonical (Anderson et al., 2018; mat).
- **FGR2R** serves as the *decomposition scaffold* because it provides fine-grained sub-instruction and sub-path structure (Hong et al., 2020).
- **NaVILA** serves as the *robotization and embodiment bridge*, since it explicitly targets legged navigation with mid-level language actions (Cheng et al., 2025; nav).
- **Touchdown** serves as the *outdoor augmentation layer* for urban navigation phrasing (Chen et al., 2019; tou).
- **CVDN** and **TEAch** serve as *interactive and task-rich extensions* rather than core route backbones (Thomason et al., 2019; cvd; Padmakumar et al., 2022; tea).

2.3 Stage A: Dataset Harvesting and Provenance Preservation

The first stage of Plan 2 is **dataset harvesting**. At this stage, the goal is not yet translation, but rather the extraction of the smallest possible set of fields needed for later normalization, decomposition, and simulation.

From **RxR**, the most useful fields are:

- instruction text,
- instruction language,
- path identifier,
- scan identifier,
- path trajectory,
- timed words or timed segments when available,
- follower metrics such as success rate, SPL, and DTW-style scores.

From **R2R**, the main fields are:

- instruction text,
- path or trajectory identifiers,
- Matterport environment linkage.

From **FGR2R**, the main fields are:

- sub-instruction segments,
- aligned sub-path information.

From **CVDN** and **TEAch**, only the navigation-relevant command turns and resolved route-bearing segments should be retained, rather than complete dialogue histories (Thomason et al., 2019; cvd; Padmakumar et al., 2022; tea). From **NaVILA**, one should harvest instruction forms and annotation structures that reveal how natural language is converted into spatially grounded, legged-navigation-compatible mid-level actions (Cheng et al., 2025; nav).

The output of Stage A should be a **provenance-preserving normalized source schema**, for example:

```
{
  "source_dataset": "rxr",
  "source_id": "instruction_26",
  "source_split": "val_seen",
  "source_language": "en",
  "instruction_raw": "...",
  "environment_type": "indoor",
  "path_available": true,
  "path_id": 11,
  "trajectory_ref": "...",
  "scan_id": "..."
}
```

This schema is important because later translations, robotized forms, and simulation results must remain traceable to an original benchmark item.

2.4 Stage B: English Normalization

Existing dataset instructions were not written for robot execution. In particular, RxR includes spoken-style multilingual route descriptions with timing information and speech transcription artifacts (Ku et al., 2020; rxr). Therefore, before translation, each source instruction should be rewritten into two English layers:

1. **canonical normalized English**,
2. **robot-natural English**.

For example:

Raw: *Okay, now you are in a room facing towards two bathtubs...*

Canonical: *Move forward through the bathroom and turn right at the doorway.*

Robot-natural: *Move forward through the bathroom. Turn right at the doorway.*

The normalization pass should preserve:

- action order,
- turns,
- landmark references,
- stop conditions,
- safety cues.

It should remove or rewrite:

- fillers,
- discourse markers,
- spoken disfluencies,
- hedges,

- unnecessary scenic narration,
- vague deixis such as *go over there* when no explicit grounding exists.

This stage is indispensable because multilingual translation quality depends heavily on the stability of the English source. Poorly normalized English produces poor multilingual commands.

2.5 Stage C: Instruction Decomposition

Every normalized instruction should be decomposed into **ordered subgoals** before translation. This is where **FGR2R** is especially useful, since it explicitly provides sub-instruction awareness (Hong et al., 2020). For example:

Walk out of the bedroom through the open door into the hallway. Turn the corner and walk into the dining area. Pass the dining table and stop near the chair.

should become:

1. exit room,
2. pass doorway,
3. enter hallway,
4. turn corner,
5. enter dining area,
6. pass table,
7. stop near chair.

The decomposition stage should attach metadata such as:

- `primitive_tags`,
- `ordered_subgoals`,
- `landmarks`,
- `turn_events`,
- `stop_event`,
- `safety_event`,
- `dialog_derived` or `route_derived`.

This decomposition is central to Plan 2 because it makes multilingual translation more reliable, helps enforce action-order preservation, and creates a clean interface for simulation-based evaluation.

2.6 Stage D: Multilingual Translation

After English normalization and decomposition are frozen, each instruction should be translated into:

- Hindi literal,
- Hindi robot-natural,
- Bengali literal,
- Bengali robot-natural,
- Telugu literal,

- Telugu robot-natural.

In this plan, **Telugu** plays a special role because RxR already includes Telugu instructions, allowing it to serve both as a target language and as a **reference calibration language** (Ku et al., 2020; rxr). **Hindi** enjoys a similar advantage. **Bengali** is the main new language without direct RxR supervision, and therefore requires stricter bilingual quality control.

The translation policy should preserve:

1. action identity,
2. action order,
3. turn direction,
4. landmark reference,
5. stop condition,
6. safety or abstention semantics.

An illustrative example is:

English : “Turn left at the door and stop near the wall.”

Hindi (literal) : दरवाज़े पर बाएँ मुड़ो और दीवार के पास रुक जाओ।

Hindi (robot-natural) : दरवाज़े तक पहुँचकर बाईं ओर मुड़ो, फिर दीवार के पास रुक जाओ।

Bengali (literal) : দরজার কাছে বাঁ দিকে ঘুরে দেওয়ালের পাশে থামো।

Bengali (robot-natural) : দরজায় পৌঁছে বাঁ দিকে ঘুরো, তারপর দেওয়ালের পাশে থামো।

Telugu (literal) : తలుపు వద్ద ఎడమవైపు తిరిగి గోడ దగ్గర ఆగండి।

Telugu (robot-natural) : తలుపు దగ్గరకు వెళ్లి ఎడమవైపు తిరిగి, తరువాత గోడ దగ్గర ఆగండి।

Each translation should be accompanied by verification fields such as:

- action_preserved,
- order_preserved,
- direction_preserved,
- landmark_preserved,
- safety_preserved.

2.7 Stage E: Robotization

Translation alone does not yield a robot-executable instruction. Therefore, Plan 2 includes a dedicated **robotization pass**, which converts human-authored or human-translated route language into a more execution-friendly imperative form. This stage is strongly motivated by NaVILA’s emphasis on transforming natural language into **mid-level language actions with spatial information** for legged navigation (Cheng et al., 2025; nav). The broader methodological motivation also aligns with work such as **VLN-Trans**, which studies the transformation of natural language instructions into more agent-friendly forms (Zhang and Kordjamshidi, 2023).

Examples:

Human-style: *Walk past the couch and kind of go toward the rug.*

Robotized: *Move past the couch. Continue toward the rug. Stop near the rug.*

Human-style: *Take the next left and head over there.*

Robotized: *Take the next left turn. Move forward to the marked target.*

Robotized instructions should satisfy the following constraints:

- at most 1–2 landmarks per clause,
- no vague pronouns such as *there*,
- no figurative expressions,
- explicit turn order,
- explicit stop condition,
- explicit abstention or caution when needed.

2.8 Stage F: Simulation and Equivalence Testing

The final stage of Plan 2 is **simulation**. This stage asks whether the translated and robotized instructions preserve the route semantics of the source instruction when executed or evaluated under shared embodied environments.

For indoor navigation, the most natural simulation backbone is the Matterport ecosystem, through **Matterport3DSimulator** and optionally **VLN-CE** for continuous-control evaluation ([mat](#); [vln](#)). For more realistic embodied-gap testing, **VLN-PE** offers a strong evaluation direction ([Wang et al., 2025](#)). For long-horizon stress testing, **LHPR-VLN** provides an especially useful benchmark ([Song et al., 2025](#); [lhp](#)).

For each underlying route instance, the following instruction variants should be compared:

1. English raw,
2. English canonical,
3. English robot-natural,
4. Hindi robot-natural,
5. Bengali robot-natural,
6. Telugu robot-natural.

The main simulation metrics should include:

- success rate,
- navigation error,
- SPL,
- DTW / nDTW / sDTW where available,
- path deviation,
- stop-point accuracy,
- landmark hit order,
- turn sequence correctness.

Since RxR explicitly distributes follower metrics including success and DTW-style measures, it is an ideal benchmark for route-preservation experiments across multilingual variants ([Ku et al., 2020](#); [rxr](#)).

2.9 Simulation Protocol and Experimental Buckets

We recommend five simulation sets.

Set A: Translation Preservation. Sample 500 source instructions:

- 250 from RxR,
- 150 from R2R,
- 100 from NaVILA-like or related embodiment-oriented data.

Compare English canonical against Hindi, Bengali, and Telugu robot-natural variants. The goal is to quantify whether multilingual translations preserve route semantics.

Set B: Robotization Gain. Sample 300 instructions and compare:

- raw original,
- normalized canonical,
- robot-natural.

The goal is to test whether robotization improves simulator-aligned execution.

Set C: Complexity Stratification. Bucket routes into short, medium, and long groups. Evaluate all languages per bucket. The goal is to understand where translation-induced drift increases with route length.

Set D: Landmark Sensitivity. Group prompts by landmark density:

- no landmark,
- one landmark,
- multiple landmarks.

The goal is to measure whether multilingual landmark grounding remains stable.

Set E: Safety and Abstention. Evaluate instructions containing blocked paths, stop-and-wait clauses, slow-move clauses, or obstacle-avoidance instructions. The goal is to verify that safety semantics survive both translation and robotization.

2.10 Recommended Corpus Sizes for Plan 2

A practical first version of Plan 2 should use the following source mix:

- **RxR:** 2,500 normalized English prompts,
- **R2R:** 1,200 prompts,
- **NaVILA-style or related embodied annotations:** 600 prompts,
- **Touchdown:** 300 prompts,
- **CVDN / TEACH navigation-relevant subset:** 400 prompts.

This yields a **5,000-prompt English normalized seed pool**. After adding Hindi, Bengali, and Telugu translations in both literal and robot-natural forms, the textual inventory expands dramatically. This is one of the major advantages of Plan 2: the source pool is finite, but the multilingual derivative corpus scales quickly.

2.11 Schema, Quality Control, and Release Format

A representative Plan 2 schema is:

```
{
  "id": "rxr_000421",
  "source_dataset": "rxr",
  "source_split": "train",
  "source_language": "en",
  "instruction_raw_en": "...",
  "instruction_canonical_en": "...",
  "instruction_robot_en": "...",
  "instruction_hi_literal": "...",
  "instruction_hi_robot": "...",
  "instruction_bn_literal": "...",
  "instruction_bn_robot": "...",
  "instruction_te_literal": "...",
  "instruction_te_robot": "...",
  "ordered_subgoals": ["exit_room", "turn_left", "move_forward", "stop_near_marker"],
  "landmarks": ["door", "marker"],
  "environment_type": "indoor",
  "safety_type": "none",
  "path_ref": "...",
  "simulation_status": "pending"
}
```

The quality-control pipeline should have three layers:

- **Linguistic QC:** remove disfluencies, normalize action order, remove vague deixis, preserve route semantics.
- **Translation QC:** verify direction, action order, landmark identity, stop condition, and safety preservation.
- **Simulation QC:** verify that translated instructions induce the same path class, and label failures such as wrong turn, missed landmark, stop failure, over-travel, under-travel, or ambiguity failure.

2.12 Step-by-Step Execution Plan

The complete execution sequence for Plan 2 should be:

1. **Select the source datasets** and assign each one a clear functional role.
2. **Harvest provenance-preserving raw data** from RxR, R2R, FGR2R, and the other chosen sources.
3. **Normalize English instructions** into canonical and robot-natural forms.
4. **Decompose instructions** into ordered subgoals and primitive tags.
5. **Translate into Hindi, Bengali, and Telugu** in literal form.
6. **Create robot-natural translations** for all three languages.
7. **Run translation QC** with bilingual reviewers.
8. **Robotize the instructions** into concise execution-oriented commands.
9. **Construct simulation sets** for route preservation, robotization gain, landmark sensitivity, complexity, and safety.
10. **Run simulation in Matterport-based environments** and stronger embodiment settings when available.

11. **Analyze failures** by type and feed the results back into normalization and robotization.
12. **Release the corpus** with provenance fields, multilingual variants, decomposition metadata, and simulation outcomes.

2.13 Summary

Plan 2 provides a complementary strategy to the Kimodo-first pipeline. Instead of beginning from a designed motion inventory, it begins from the best available embodied language datasets, then transforms them into a multilingual, robot-executable, simulation-tested instruction corpus. RxR provides the strongest multilingual backbone; R2R provides compact English support; FGR2R provides decomposition structure; NaVILA provides embodiment-aware robotization cues; Touchdown adds outdoor phrasing; CVDN and TEACH extend the design toward interaction and task richness; and Matterport-based simulators provide the evaluation substrate. The most powerful outcome of Plan 2 is not just a translated dataset, but a **translation-preserving, robotization-aware, simulation-validated multilingual instruction resource** that can later be fused with the Kimodo-derived motion corpus in Plan 1.

Suggested immediate next step. The best next deliverable is a **master normalization and robotization guideline table** specifying, for each source dataset, which fields are extracted, how English is normalized, how subgoals are derived, what translation rules apply, and how simulation success or failure is recorded.

3 Fine-Tuning OpenVLA with the Two Pragya Data Streams: Platform, Data Conversion, Training Strategy, Hardware, and Budget

This section presents a detailed plan for adapting **OpenVLA** to the two data streams proposed earlier: **(i)** the Kimodo-derived multilingual motion corpus and **(ii)** the existing-datasets-derived multilingual route-and-navigation corpus built from resources such as RxR, R2R, and NaVILA-like annotations. The central conclusion is straightforward: these two resources are *not* directly consumable by OpenVLA in their raw form. OpenVLA is a **vision-language-action** model whose fine-tuning pipeline expects robot episodes with **observations, language, and action supervision**, typically in **RLDS-style episodic format** or via a custom dataset wrapper compatible with the official training code (`ope`, `c,a`; `oxe`). Therefore, the practical challenge is not only model fine-tuning, but **data transduction**: the two corpora must first be converted into image-conditioned, temporally aligned, action-labeled robot episodes before OpenVLA adaptation becomes meaningful.

The most important design decision is that OpenVLA should *not* be asked, in the first iteration, to directly predict low-level continuous joint commands for humanoid or quadruped navigation from translated route text alone. Instead, the highest-probability path is to use OpenVLA as a **high-level action-chunk policy**: given robot observations and a multilingual instruction, OpenVLA predicts a short, structured action chunk such as `forward_0.4m`, `turn_left_30deg`, `stop`, `sidestep_right`, or `step_over`. A separate low-level locomotion controller or simulator policy then executes that chunk. This separation reduces action-space mismatch, improves data efficiency, and aligns more naturally with the structure of both the Kimodo-derived motion vocabulary and the navigation-derived route language (`ope`, `c,b`).

3.1 Why the Two Datasets Cannot Be Used Directly

The two proposed datasets are extremely valuable, but they are **semantically incomplete** from the perspective of OpenVLA.

Dataset Stream 1: Kimodo-derived corpus. The Kimodo-based corpus provides a multilingual library of motion intents, posture transitions, obstacle-related commands, and short compositional action phrases. This corpus is excellent for defining a **motion ontology**, i.e., a structured inventory of movement primitives and compositional motor semantics. However, by itself it does not provide the full OpenVLA input-output structure, because it does not necessarily contain:

- synchronized camera observations,
- robot proprioception aligned with execution,

- per-step robot actions in OpenVLA-compatible format,
- episode-level state transitions under realistic embodiment.

Dataset Stream 2: Existing-datasets-derived corpus. The RxR/R2R/NaVILA-style corpus provides route instructions, landmark references, turn sequences, stop conditions, and safety semantics. This makes it excellent for defining a **navigation language ontology**. But again, it does not directly provide the full VLA supervision tuple required by OpenVLA. In most cases, these datasets are language-route datasets rather than robot-control datasets. Even when path information is available, OpenVLA still needs:

- actual visual observations,
- aligned action labels or action chunks,
- episode packaging,
- embodiment-specific controller behavior.

Therefore, the correct framing is:

Kimodo-derived data contributes motion semantics; route-derived data contributes navigation semantics; OpenVLA fine-tuning requires a fused observation-language-action dataset constructed from both.

3.2 What OpenVLA Expects

OpenVLA is a large **7B vision-language-action model** built for robot action prediction from visual observations and language commands, and it is trained over large-scale robot episodes from **Open X-Embodiment**. The official project and repository describe a fine-tuning workflow centered around robot episodes, and the official codebase supports lightweight adaptation methods such as **LoRA** and **OFT** in addition to more expensive full fine-tuning ([ope](#), [c](#), [a](#), [b](#)). In practice, this means that each training example must be embedded inside an **episode** with the following conceptual structure:

$$(\mathbf{o}_1, \mathbf{a}_1), (\mathbf{o}_2, \mathbf{a}_2), \dots, (\mathbf{o}_T, \mathbf{a}_T), \quad \text{conditioned on instruction } x,$$

where:

- $\mathbf{o}_t$ includes one or more images and optionally proprioception,
- x is the instruction (English, Hindi, Bengali, Telugu, or a multilingual pack),
- $\mathbf{a}_t$ is the target action label or action tokenization for that time step or short chunk.

Thus, the first engineering task is to convert the two corpora into a **shared episodic schema**. If this conversion step is weak, then multilingual fine-tuning will appear to fail even when the root problem is actually poor action-space design rather than poor language learning.

3.3 Recommended Overall Architecture

The safest and most practical strategy is to fine-tune OpenVLA as a **high-level action-chunk predictor** rather than as a raw joint-space controller. In this design:

1. OpenVLA takes as input a current robot observation and a multilingual instruction.
2. OpenVLA outputs a **small action chunk** or short action plan.
3. A separate low-level locomotion or embodiment controller executes that chunk.

This design is preferable for several reasons:

- it reduces the complexity of the output space,

- it allows the same high-level instruction semantics to be reused across humanoid and quadruped settings,
- it lets the Kimodo-derived motion vocabulary and the translated route vocabulary meet at a common interface,
- it is more data-efficient than full low-level imitation,
- it is more compatible with simulator-first training.

A suitable **high-level action chunk vocabulary** for the first version is:

- forward_small, forward_medium, forward_large,
- turn_left_small, turn_left_large,
- turn_right_small, turn_right_large,
- sidestep_left, sidestep_right,
- stop, wait,
- slow_move, inspect,
- lower_body, raise_body,
- step_over, go_around_left, go_around_right.

This chunk vocabulary should be derived from:

- the **Kimodo-derived motion primitives** for posture, stepping, obstacle handling, and gesture,
- the **route-derived instruction corpus** for navigation, turns, stop conditions, and landmark progression.

3.4 How the Two Datasets Should Be Fused

The two data streams should be fused asymmetrically.

Role of the Kimodo-derived corpus. This corpus should define the **action ontology**. It contributes:

- motion primitive definitions,
- posture transitions,
- obstacle-handling verbs,
- embodied action synonyms across languages,
- compact action chunk design.

Role of the route-derived corpus. This corpus should define the **navigation language ontology**. It contributes:

- turn structure,
- landmark references,
- multi-stage route composition,
- stop and wait semantics,
- safety and abstention cues,
- multilingual route phrasing.

Final fused OpenVLA dataset. The final training dataset should consist of episodes in which each example contains:

- camera observation(s),
- multilingual instruction(s),
- current route state or progress,
- action chunk label(s),
- optional proprioception,
- optional safety state flags.

Thus, the fusion logic is:

Kimodo tells the model what motion chunks mean; RxR/R2R/NaVILA-style corpora tell the model what route language means; the simulator generates the aligned episodes that OpenVLA actually learns from.

3.5 Data Conversion Pipeline to RLDS/OpenVLA Format

The dataset conversion pipeline should have six stages.

Stage 1: Instruction normalization. Take the raw navigation instruction and produce:

- canonical English,
- robot-natural English,
- literal Hindi/Bengali/Telugu,
- robot-natural Hindi/Bengali/Telugu.

Stage 2: Subgoal decomposition. Convert each route instruction into ordered subgoals, e.g.:

exit room → turn left → move forward → stop near marker

Stage 3: Action-chunk mapping. Map each subgoal to one or more action chunks from the motion ontology. For example:

turn left → turn_left_small
move forward to marker → forward_medium + stop

Stage 4: Simulation rollout generation. Execute the route or task in simulation and record:

- RGB frames,
- optional depth or extra views,
- proprioception if used,
- action chunks,
- task progress,
- success/failure status.

Stage 5: RLDS packaging. Convert the rollout into an RLDS-style episode or a custom dataset wrapper compatible with the OpenVLA fine-tuning code ([ope](#), [a](#); [oxe](#)).

Stage 6: Multilingual episode augmentation. For the same episode, attach multiple instruction variants:

- English robot-natural,
- Hindi robot-natural,
- Bengali robot-natural,
- Telugu robot-natural.

This yields multilingual supervision over the same embodied trajectory.

3.6 Recommended Training Schema

A practical episode-level schema is shown below.

```
{
  "episode_id": "rxr_g1_000421",
  "robot_type": "g1",
  "environment_type": "indoor",
  "source_dataset": "rxr",
  "instruction": {
    "en_robot": "...",
    "hi_robot": "...",
    "bn_robot": "...",
    "te_robot": "..."
  },
  "observations": {
    "image_front": "...",
    "image_aux": "...",
    "proprio": "..."
  },
  "route_state": {
    "subgoal_index": 2,
    "landmarks_remaining": ["door", "marker"],
    "safety_flag": "none"
  },
  "actions": [
    "forward_small",
    "turn_left_small",
    "forward_medium",
    "stop"
  ],
  "success": true,
  "metadata": {
    "split": "train",
    "difficulty": "medium"
  }
}
```

For a timestep- or chunk-level version, each training item can contain:

- current image,
- optional previous image(s),
- multilingual instruction,

- current subgoal state,
- next action chunk,
- termination or stop flag.

3.7 Platform Recommendations

The platform stack should be divided into three layers: **data curation**, **simulation**, and **training**.

Data curation layer. Use Python preprocessing, JSON/Parquet metadata, and RLDS packaging tools. This layer handles:

- route extraction,
- instruction normalization,
- translation integration,
- action-chunk mapping,
- episode packaging.

Simulation layer. For route-language benchmarking and episode generation, use:

- **Matterport3DSimulator** for R2R-style evaluation ([mat](#)),
- **VLN-CE** for continuous-control settings ([vln](#)),
- **Isaac Sim / Isaac Lab** if the target embodiment is G1-like humanoid or legged robotics and you want higher-fidelity embodied control.

A pragmatic strategy is:

1. first validate route preservation in Matterport-based environments,
2. then generate richer embodiment-aligned rollouts in Isaac-based environments,
3. then package them for OpenVLA.

Training layer. Use the official **OpenVLA** fine-tuning stack ([ope](#), [a](#), [c](#)). The recommended order is:

- first use **LoRA** or **OFT**,
- avoid full fine-tuning initially,
- run English-only pilots first,
- then add Hindi, Bengali, and Telugu.

3.8 Recommended Training Phases

We recommend five phases.

Phase 0: Data engineering only. Before any OpenVLA fine-tuning, complete:

- action-chunk ontology design,
- multilingual instruction normalization,
- simulator rollout generation,
- RLDS dataset packaging.

Phase 1: English-only pilot. Train OpenVLA to predict action chunks using only the English robot-natural instructions. The goal is to validate the action interface before multilingual expansion.

Setup Tier	Suggested Hardware	Recommended Use	Expected Scope
Pilot	1 × A100 80GB	LoRA; English-only or small multilingual pilot; action-chunk validation	one embodiment, small ablations
Research base-line	4 × A100 80GB or 4 × H100 80GB	OFT or stronger LoRA runs; multilingual adaptation; multiple seeds	one solid paper-quality campaign
Full campaign	8 × A100/H100 80GB	multi-run ablations, multilingual + embodiment branches, stronger scaling	serious lab-scale experimentation
Full fine-tune regime	8 × A100/H100 80GB (or more)	only if action interface is already validated and parameter-efficient tuning is insufficient	highest cost, lowest initial priority

Table 3: Practical hardware tiers for OpenVLA adaptation based on official OpenVLA and OpenVLA-OFT guidance.

Phase 2: Multilingual expansion. Add Hindi, Bengali, and Telugu instruction variants for the same underlying episodes. This tests whether multilingual instructions map to the same action predictions.

Phase 3: Safety and abstention adaptation. Add episodes involving stop, wait, inspect, obstacle negotiation, and slower motion. This phase is critical for deployment-oriented behavior.

Phase 4: Embodiment branching. Create separate adapters or checkpoints for:

- humanoid,
- quadruped,
- optionally a shared high-level policy with separate low-level controllers.

Phase 5: Long-horizon and outdoor generalization. Add longer route episodes and street-scale language once the shorter indoor instruction pipeline is stable.

3.9 Hardware Recommendations

The OpenVLA repository and the OpenVLA-OFT project strongly suggest that **parameter-efficient fine-tuning** is the correct default for most research users. The official materials indicate that:

- **LoRA fine-tuning** is feasible on a **single A100 80GB** and can be made to work on smaller-memory GPUs with aggressive batch-size tuning and gradient accumulation,
- **OFT** experiments are commonly run on **4 or 8 A100/H100 80GB GPUs** for roughly **50k–150k gradient steps**,
- **full fine-tuning** is substantially more expensive and generally requires a full multi-GPU node ([ope](#), [a](#),[b](#)).

Table 3 summarizes a practical planning guide.

3.10 Actual Cloud Cost Numbers

To make budget planning concrete, we now instantiate the hardware discussion using currently listed cloud GPU prices. On **Runpod**, the posted on-demand rates include:

- **A100 PCIe 80GB**: \$1.19/hr,
- **A100 SXM 80GB**: \$1.39/hr,
- **H100 PCIe 80GB**: \$1.99/hr,
- **H100 SXM 80GB**: \$2.69/hr ([run](#)).

On **CoreWeave**, the posted rates include:

- **A100 80GB PCIe/NVLink**: \$2.21/hr,
- **H100 PCIe**: \$4.25/hr,

Scenario	Configuration	GPU Cost	Interpretation
One-day pilot	1 × Runpod A100 PCIe 80GB for 24h	\$28.56	cheapest serious LoRA pilot
One-day pilot	1 × Runpod A100 SXM 80GB for 24h	\$33.36	slightly faster/more premium A100 pilot
One-day pilot	1 × Runpod H100 PCIe 80GB for 24h	\$47.76	stronger but still reasonable single-GPU pilot
One-day pilot	1 × CoreWeave A100 80GB for 24h	\$53.04	more expensive A100 pilot
Two-day research run	4 × Runpod A100 SXM 80GB for 48h	\$266.88	good multilingual OFT/LoRA run
Two-day research run	4 × Runpod H100 PCIe 80GB for 48h	\$382.08	faster, stronger run for heavier experiments
Two-day research run	4 × CoreWeave A100 80GB for 48h	\$424.32	solid but pricier A100 baseline
Two-day research run	4 × CoreWeave H100 PCIe for 48h	\$816.00	high-end multilingual run
Three 48h A100 runs	4 × Runpod A100 SXM 80GB, repeated 3 times	\$800.64	realistic paper-quality campaign
Three 48h A100 runs	4 × CoreWeave A100 80GB, repeated 3 times	\$1,272.96	pricier paper-quality campaign
Three 48h H100 runs	4 × Runpod H100 PCIe 80GB, repeated 3 times	\$1,146.24	strong H100 campaign
Three 48h H100 runs	4 × CoreWeave H100 PCIe, repeated 3 times	\$2,448.00	premium H100 campaign

Table 4: Actual GPU-only cloud cost calculations using currently listed Runpod and CoreWeave hourly prices. These estimates do not include storage, egress, failed runs, simulator generation, or long-term checkpoint retention.

- **AI Object Storage:** \$0.110/GB/month ([cor](#)).

Using those posted prices, Table 4 provides actual GPU-only cost estimates for common OpenVLA scenarios.

The main practical consequence is:

- a true **single-GPU LoRA pilot** is only on the order of **\$30–\$55 per day**,
- a realistic **4-GPU A100 research run for 48 hours** is on the order of **\$267–\$424**,
- a **three-run multilingual campaign** on 4 A100s is roughly **\$800–\$1,300 GPU-only**,
- a similar **4-H100 campaign** is roughly **\$1,146–\$2,448 GPU-only**,
- storage and simulator generation are additional costs.

3.11 Storage, Data Volume, and Operational Specs

Storage planning is often underestimated. A serious OpenVLA adaptation pipeline must budget for:

- raw benchmark datasets,
- multilingual metadata,
- simulator rollouts,

- RLDS episode shards,
- checkpoints,
- evaluation videos,
- ablation outputs and logs.

A realistic first research campaign should assume:

- **2–5 TB** storage for a serious pilot with multi-view image rollouts,
- more if high-resolution video or multiple embodiments are stored simultaneously.

Using CoreWeave’s listed object storage price of **\$0.110/GB/month**, this implies:

- **2 TB $\approx$ \$220/month**,
- **5 TB $\approx$ \$550/month** (cor).

For operational reproducibility, every run should record:

- dataset version hash,
- translation pack version,
- action ontology version,
- simulator version,
- control-policy version,
- OpenVLA checkpoint base,
- adapter type (LoRA/OFT),
- GPU configuration,
- learning rate, batch size, gradient accumulation, and sequence horizon.

3.12 Budget Planning

Cloud cost depends strongly on provider, region, reserved versus on-demand pricing, and whether simulation and training are run on the same stack. Therefore, it is more robust to budget in **compute bands** than in a single fixed dollar number.

Band A: low-cost pilot.

- $1 \times \text{A100 80GB}$,
- LoRA only,
- English-first pilot,
- small simulator-generated RLDS set,
- one embodiment,
- one or two full training runs.

Using current listed prices, two one-day A100 runs imply roughly:

- **Runpod A100 PCIe:** $2 \times \$28.56 = \mathbf{\$57.12}$,
- **CoreWeave A100 80GB:** $2 \times \$53.04 = \mathbf{\$106.08}$ (run; cor).

After adding storage, logging, and a small rerun buffer, a good practical planning number is **\$150–\$250 total**.

Band B: standard research campaign.

- 4 × A100 80GB or equivalent,
- OFT or stronger LoRA,
- multilingual runs,
- safety/abstention episodes,
- multiple seeds and ablations,
- medium-scale simulation.

A concrete three-run, 48-hour-per-run budget is:

- **Runpod 4×A100 SXM:** $3 \times \$266.88 = \text{\$800.64}$,
- **CoreWeave 4×A100:** $3 \times \$424.32 = \text{\$1,272.96}$ (run; cor).

Once storage, simulator costs, debugging runs, and failed jobs are added, a realistic planning band is **\$1.5k–\$3k total**.

Band C: stronger H100 campaign.

- 4 × H100 80GB,
- multilingual runs,
- larger ablation grid,
- faster turnaround,
- stronger scaling experiments.

A concrete three-run, 48-hour-per-run budget is:

- **Runpod 4×H100 PCIe:** $3 \times \$382.08 = \text{\$1,146.24}$,
- **CoreWeave 4×H100 PCIe:** $3 \times \$816.00 = \text{\$2,448.00}$ (run; cor).

After storage, simulation, and a rerun buffer, a realistic planning band is **\$2k–\$4.5k total**.

The strongest recommendation is:

Do not spend H100 or full-fine-tuning budget before validating the action-chunk representation on a smaller A100-based LoRA/OFT pilot.

3.13 Recommended Concrete Plan for Pragya

A concrete, high-probability sequence for your setting is:

Version 1. Use the route-derived multilingual corpus (RxR + R2R + selected NaVILA-style samples), create simulator rollouts, map route subgoals into action chunks, package into RLDS, and fine-tune OpenVLA with **LoRA** on **one embodiment**. Keep the action vocabulary small.

Version 2. Inject the Kimodo-derived motion corpus to enrich the action ontology with posture changes, obstacle handling, and richer movement primitives. Continue with adapter-based training.

Version 3. Branch into separate adapters for:

- humanoid navigation and posture-rich tasks,
- quadruped or legged route following and obstacle negotiation.

Version 4. Add longer route episodes, stronger safety signals, and outdoor phrasing.

3.14 Biggest Technical Risk

The biggest technical risk is **not compute**. It is **action-space mismatch**. If the action chunks are poorly designed, or if the same multilingual instruction maps to inconsistent action interfaces across episodes, then OpenVLA fine-tuning will appear unstable regardless of hardware scale. Therefore, the correct order is:

1. define the action-chunk ontology,
2. generate RLDS episodes,
3. validate English-only imitation,
4. add multilingual variants,
5. scale hardware and ablations.

3.15 Summary

The correct way to fine-tune OpenVLA with the two proposed datasets is *not* to feed them directly into the model. Instead, the two corpora should be fused into a simulator-generated, multilingual, action-labeled episode dataset. The Kimodo-derived corpus should define the motion vocabulary; the existing-datasets-derived corpus should define the route and landmark vocabulary; and simulation should generate the observation-action alignments that OpenVLA actually requires. The most practical training setup is to use OpenVLA as a **high-level action-chunk predictor** and leave low-level locomotion to a separate controller. For hardware, start with **LoRA/OFT on 1–4 A100 80GB GPUs** before considering any full fine-tuning. For budget, the current real-world planning numbers are roughly:

- **\$150–\$250** for a serious initial pilot,
- **\$1.5k–\$3k** for a good A100-based multilingual campaign,
- **\$2k–\$4.5k** for a stronger H100-based campaign.

And for engineering priority, treat **action interface design** as more important than raw compute scale during the first iteration.

3.16 Suggested Immediate Next Deliverable

The best immediate next step is to write a **formal RLDS/OpenVLA schema specification** for your fused dataset, including:

- multilingual instruction fields,
- route-state fields,
- action-chunk vocabulary,
- simulator metadata,
- embodiment metadata,
- train/dev/test split policy.

That schema should be finalized before any GPU budget is committed.

References

- Coreweave gpu cloud pricing. <https://www.coreweave.com/gpu-cloud-pricing>.
- Cvdm official repository. <https://github.com/mmurray/cvdm>.
- Lhpr-vln dataset card. <https://huggingface.co/datasets/Starry123/LHPR-VLN>.
- Matterport3dsimulator. <https://github.com/peteanderson80/Matterport3DSimulator>.
- Navila project page. <https://navila-bot.github.io/>.
- Open x-embodiment repository. https://github.com/google-deepmind/open_x_embodiment.
- a. Openvla official repository. <https://github.com/dusty-nv/openvla>.
- b. Openvla-ofc project page. <https://openvla-ofc.github.io/>.
- c. Openvla project page. <https://openvla.github.io/>.
- Quart / quar-vla project page. <https://quart-robot.github.io/>.
- Reverie official repository. <https://github.com/YuankaiQi/REVERIE>.
- Runpod gpu pricing. <https://www.runpod.io/gpu-pricing>.
- Rxr official repository. <https://github.com/google-research-datasets/RxR>.
- Soon / fao project page. <https://scenario-oriented-object-navigation.github.io/>.
- Teach official repository. <https://github.com/alexa/teach>.
- Touchdown official repository. <https://github.com/lil-lab/touchdown>.
- Vln-ce repository. <https://github.com/jacobkrantz/VLN-CE>.
- Peter Anderson, Qi Wu, Damien Teney, Jake Bruce, Mark Johnson, Niko Sünderhauf, Ian Reid, Stephen Gould, and Anton van den Hengel. 2018. Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments. In *CVPR*.
- Howard Chen, Alane Suhr, Dipendra Misra, Noah Snaveley, and Yoav Artzi. 2019. Touchdown: Natural language navigation and spatial reasoning in visual street environments. In *CVPR*.
- Anjie Cheng et al. 2025. Navila: Legged robot vision-language-action model for navigation. In *RSS*.
- Yan Ding et al. 2024. Quar-vla: Vision-language-action model for quadruped robots. In *ECCV*.
- Yicong Hong, Qi Wu, Yuankai Qi, Doheon Park, et al. 2020. Sub-instruction aware vision-and-language navigation. In *EMNLP*.
- Alexander Ku, Peter Anderson, Roma Patel, Eugene Ie, and Jason Baldridge. 2020. Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding. In *EMNLP*.
- Aishwarya Padmakumar, Jesse Thomason, Ashish Shrivastava, et al. 2022. Teach: Task-driven embodied agents that chat. In *AAAI*.
- Yuankai Qi, Qi Wu, et al. 2020. Reverie: Remote embodied visual referring expression in real indoor environments. In *CVPR*.
- Hao Song et al. 2025. Towards long-horizon vision-language navigation: Platform, benchmark and method. *arXiv preprint arXiv:2412.09082*.
- Jesse Thomason, Michael Murray, Maya Cakmak, and Luke Zettlemoyer. 2019. Vision-and-dialog navigation. *arXiv preprint arXiv:1907.04957*.

Yikai Wang et al. 2025. Rethinking the embodied gap in vision-and-language navigation: A holistic study of physical and visual disparities. *arXiv preprint arXiv:2507.13019*.

Yichi Zhang and Parisa Kordjamshidi. 2023. Vln-trans: Translator for the vision and language navigation agent. In *ACL*.